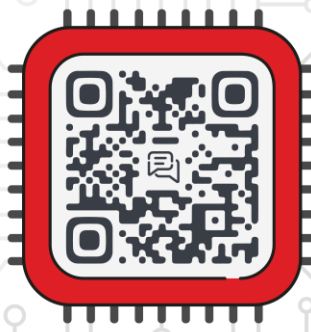




6



6



720 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

Thread

د. عملي مشترك



برمجة ٣

RB Informatics; 27/11/2025

مرحباً اليوم موضوعنا عن ال Thread

مفهوم ال Thread

ال Thread هو خط (مسار) تنفيذ مستقل داخل البرنامج (بإمكاننا تشبيهه كوحدة المعالجة المركزية الافتراضية تقريباً) ، حيث يمكننا اعتباره مسار يمكن لل CPU جدولة تشغيله .

يستخدم ال Thread :

1. لعمل أكثر من وظيفة بنفس الوقت.

2. لكي لا نجعل البرنامج يعلق .

3. لكي نعالج عمليات كبيرة دون أن يتوقف البرنامج.

مثل: تحميل ملف بالخلفية _ تشغيل موسيقى أثناء استخدام التطبيق



ال Thread يملك عدة دوال و خصائص تجعلنا قادرين على التحكم بطريقة تنفيذ هذه الأوامر.

الJVM تسمح للتطبيق بتشغيل عدة Threads في وقت واحد.

حيث كل thread ينفذ أجزاء من الكود بالتوازي مع ال (main thread) ولكل thread أولوية خاصة.

في جافا ، ال JVM لا تضمن تنفيذ الThreads بنفس الأسلوب في كل مرة لأن طريقة الجدولة تعتمد على نظام التشغيل و تختلف من حاسوب الى آخر .

■ سنقوم بشرح ال Thread بمثال مبسط لضمان فهم الفكرة:

تخيل أنك ذهبت إلى متجر العم سعيد لشراء سندويشة فلافل. عمو سعيد يعمل وحده، لذلك يقوم بكل المهام بشكل متتال : تحضير الساندويش ، ثم لفّها، ثم عدّ المال... وهكذا. هذا يعني أنه سينهي كل المهام بالمدة الإجمالية لجميع الأعمال معاً.

لكن لو كان لدى عمو سعيد عاملان يساعدانه، فيمكن لكل واحد أن ينفذ مهمة مختلفة في نفس الوقت. وهكذا يصبح الزمن الكلي مساوياً لأطول مهمة فقط

فمثلاً: إذا كان تحضير السندويشة يحتاج 9 دقائق، وعدّ المال يحتاج 3 دقائق، فعمو سعيد وحده سينتهي خلال 12 دقيقة

أما بوجود أكثر من عامل يعملون بالتوازي، فيمكن إنهاء المهام خلال 9 دقائق فقط. "والآن بعد ان انتهيت من قراءة القصة الصغيرة لتتعلم كيف ننشئ ال Thread وكيف نشغله .

خطوات انشاء Thread وتشغيله:

هناك طريقتان رئيسيتان:

1. إما أن نرث من كلاس اسمه Thread

```
class MyThread extends Thread {
    @Override
    public void run() {
        //التعليمات التي نريد تنفيذها.
    }
}
```

```
public class Main {
    public static void
    main(String[]args) {
        MyThread t = new MyThread();
        t.start();
        //تشغيل ال Thread
    }
}
```

ملاحظة:

لا نستخدم run() مباشرة بل نستخدم start() لأنه ينشئ Thread جديد و يستدعي run() داخلياً .

2. او نستخدم Interface اسمه Runnable

هذه الطريقة افضل لأن Java لا تدعم الوراثة المتعددة .

```
class MyTask implements
Runnable {
    @Override
    public void run() {
    }
}
```

```
public class Main{
    public static void main (String[]args) {
        Thread t = new Thread(new MyTask());
        t.start();
    }
}
```

التوابع الخاصة بكلاس ال Thread

1. public void start() : تستخدم لتشغيل كائن ال Thread أي تنفيذ الأوامر التي في الدالة run() وترمي استثناء من نوع `IllegalThreadStateException` في حال كان ال Thread يعمل قبل استدعائه.

2. public void run() : يتم تنفيذ التعليمات داخل تابع run() فور تشغيل ال thread

3. `public static void sleep()` : توقف تنفيذ تعليمات ال thread التي قامت باستدعائها لمدة محددة ثم يعود للتنفيذ بعد انتهاء المدة
4. `public void join()` : توقف البرنامج من تنفيذ التعليمات الموجودة ضمن تابع ال main حتى تنتهي جميع التعليمات الموجودة في هذه ال thread .
5. `public void setPriority()` : لتحديد اولوية ال thread .
6. `public void isAlive()` : يخبرك اذا ال thread مازال يعمل ام لا .

🔗 تمرين اضافي :

- اكتب برنامج يقوم بتعريف مهمة باسم factorial تقوم بطباعة قيم العاملية للأعداد ضمن مجال معطى بحيث يتواجد فارق ثانية بين طباعة كل قيمة و الاخرى ثم تنفيذ مهمتين ضمن البرنامج العام ، تقسمان العمل فيما بينهما لطباعة العاملية بين a و b مدخلات مع مراعاة انتهاء الاولى ثم بدء الثانية .

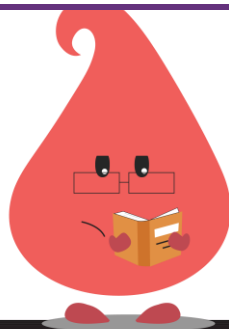
الحل :

```
class Factorial implements Runnable {
    private int start;
    private int end;
    public Factorial(int start, int end) {
        this.start = start;
        this.end = end;
    }
    private long factorial(int n) {
        long result = 1;
        for (int i = 2; i <= n; i++) {
            result *= i;
        }
        return result;
    }
    @Override
    public void run() {
        for (int i = start; i <= end; i++) {
            System.out.println("Factorial of " + i + " = " + factorial(i));
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {}
        }
    }
}
```

```
public class Main {  
    public static void main(String[] args) throws InterruptedException {  
        int a = 1;  
        int b = 6;  
  
        Factorial task1 = new Factorial(a, 3);  
        Factorial task2 = new Factorial(4, b);  
        Thread t1 = new Thread(task1);  
        Thread t2 = new Thread(task2);  
        t1.start();  
        try {  
            t1.join();  
        } catch (InterruptedException e) {  
            System.out.println("Interrupted");  
        }  
        t2.start();  
        try {  
            t2.join();  
        } catch (InterruptedException e) {  
            System.out.println("Interrupted");  
        }  
  
        System.out.println("Done!");  
    }  
}
```

تصريح :

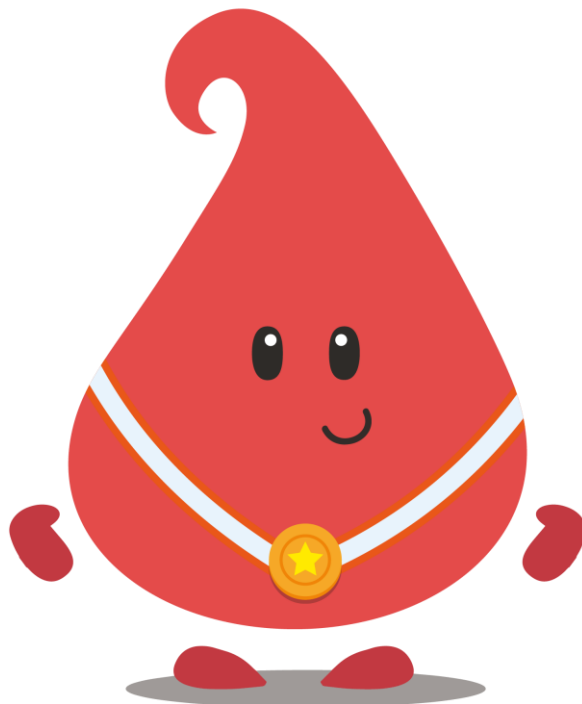
- اكتب برنامج يقوم بإنشاء مهمتين الأولى تدور في حلقة عشر مرات ومن أجل كل دورة تقوم بطباعة عدد صحيح من 0 حتى 99 ثم تنام لمدة ثانية واحدة ، أما المهمة الثانية فتقوم بنفس العمل ولكن مع طباعة عدد حقيقي عشوائي بدلا من عدد صحيح .



```
1 import java.util.Random;
2 class DoubleTask implements Runnable {
3     private String taskName;
4     private final int MAX_ITERATIONS = 10;
5     private Random random = new
Random();
6     public DoubleTask(String name) {
7         this.taskName = name;
8         System.out.println("Task created: "
+ taskName);
9     }
10    @Override
11    public void run() {
12        System.out.println("\nStarting Task: "
+ taskName);
13        try {
14            for (int i = 0; i <
MAX_ITERATIONS; i++) {
15                double randomDouble =
random.nextDouble();
16                System.out.println(taskName + " - Iteration
" + (i + 1) + ": Random Double is " +
randomDouble);
17                Thread.sleep(1000);
18            }
19        } catch (InterruptedException e) {
20            System.out.println("Task
interrupted: " + taskName);
21        }
22        System.out.println("Task finished: "
+ taskName);
23    }
24 }
```

```
1 import java.util.Random;
2 class IntegerTask implements Runnable {
3     private String taskName;
4     private final int MAX_ITERATIONS = 10;
5     private Random random = new
Random();
6     public IntegerTask(String name) {
7         this.taskName = name;
8         System.out.println("Task created: " +
taskName);
9     }
10    @Override
11    public void run() {
12        System.out.println("\nStarting Task: "
+ taskName);
13        try {
14            for (int i = 0; i < MAX_ITERATIONS;
i++) {
15                int randomNumber =
random.nextInt(100);
16                System.out.println(taskName + " - Iteration "
+ (i + 1) + ": Random Integer is " +
randomNumber);
17                Thread.sleep(1000);
18            }
19        } catch (InterruptedException e) {
20            System.out.println("Task
interrupted: " + taskName);
21        }
22        System.out.println("Task finished: " +
taskName);
23    }
24 }
```

```
1 public class Main {
2     public static void main(String[] args) {
3         Thread thread1 = new Thread(new
IntegerTask("Integer_Task"));
4         Thread thread2 = new Thread(new DoubleTask("Double_Task"));
5         thread1.start();
6         thread2.start();
7         try {
8             thread1.join();
9             thread2.join();
10        } catch (InterruptedException e) {
11            System.out.println("Main thread interrupted.");
12        }
13        System.out.println("\nMain Program finished.");
14    }
15 }
```



THE END